Overview of Recent Generative Models

Amirhossein Mohammadi

2 May 2024

Abstract

In this document, we will review some of the methods for generative modeling that we covered throughout the semester. Specifically, I will talk about normalizing flows, variational auto encoders (VAEs) and diffusion models as latent variable models, flow models, and finally finish with talking about guidance briefly. I appreciate any thoughts or feedback you have about the document.

Normalizing Flows

Generative modeling in its most raw form can be described as follows. Given that you observe a set of observations $\{x_i\}_{i=1}^N$ where $x_i \in \mathbb{R}^n$, the goal is to generate new samples that closely resemble these observations.

One natural way of thinking about generation process is to model it using a parameterized function $f_\theta(\cdot)$, where outputs of this function give us samples similar to our observations. Let's first define an input to this function and set a simple objective.

We can set the input to the function f_θ as some samples $z \in \mathbb{R}^m$ from a tractable initial distribution, say $z \sim p_{\text{init}}$ where $p_{\text{init}} = \mathcal{N}(0, I)$. Now the function f_θ is defined as follows: $f_\theta : \mathbb{R}^m \to \mathbb{R}^n$, $z \mapsto x$. Naturally, we model the observed data as random variables; these random variables follow the distribution $x_i \sim p_{\text{data}}$, and we aim to find good proposals generated by $f_\theta(z)$ or $\hat{x}$ where we assume they follow $\hat{x} \sim p_{\text{model}}$.

Given that our only clue is some observations x_i, the first natural idea that comes to mind is to maximize the probability of the observed data under our model, or equivalently its log probability:

$$\max_\theta \sum_{i=1}^{N} \log p_{\text{model}}(x_i; \theta) \tag{1}$$

You might ask what p_{model} is precisely? p_{model} can be mathematically derived given that f_θ is invertible and z are sampled from a tractable initial distribution:

$$p_{\text{model}}(x; \theta) = p_{\text{init}}\big(f_\theta^{-1}(x)\big) \left|\det\big(\tfrac{\partial\, f_\theta^{-1}(x)}{\partial x}\big)\right| \tag{2}$$

This is exactly what *normalizing flows* aim to achieve. They try to solve the optimization problem (1) to find θ that gives the highest likelihood under the model probability distribution. Notice that f_θ itself can consist of a series of simple functions, meaning $f_\theta(z) = g_k\big(g_{k-1}(\dots g_1(z)\dots)\big)$.

If we want to train our model to learn a rich mapping, our choice of g_i blocks must be such that overall f_θ be sufficiently expressive, yet each block's Jacobian (or its inverse) must be efficiently computable to optimize eq. (1). In practice, this is achieved by choosing g_i simple operations—such as affine layers or point wise one-to-one nonlinearities—that allow a tractable determinant computation for f_θ.

$$\log \left|\det \frac{\partial\, f_\theta^{-1}(x)}{\partial x}\right| = \sum_{i=1}^{k} \log \left|\det \frac{\partial\, g_i^{-1}(z_i)}{\partial z_i}\right|, \tag{3}$$

More sophisticated flows incorporate invertible 1×1 convolutions to permute channel dimensions and capture global dependencies, or employ locally structured transforms reminiscent of self-attention mechanisms, thereby enhancing the model's ability to learn complex spatial correlations without sacrificing computational tractability[1].

Latent Variable Models

VAEs

At the end of the day f_θ must be an invertible function and find parameters for this model. However this is a big bottleneck as it is hard to find an expressive mapping while maintaining invertibility, and also remain efficient. If we could get rid of invertibility property of f_θ, we could use our deep neural nets which are *universal approximators* without worrying about expressivity and efficiency. So, basically we write log probability of observation under our model f_θ and try to maximize that:

$$\max_\theta \log p_{\text{model}}(\{x_i\}_{i=1}^N; \theta) = \max_\theta \sum_{i=1}^{N} \log \frac{p_{\text{model}}(x_i; \theta)}{\displaystyle\int_{\mathcal{X}} p_{\text{model}}(x; \theta)\; dx} \tag{4}$$

However if we lose the bijectivity property of f_θ the denominator in (4) becomes a huge problem as the integral is intractable and it's not *normalized* any more. Yet there is a very interesting observation here. Each z is mapped to exactly one $\hat{x}$. If we think of it this way, we can do a trick: we can assume that x_i *is generated based on some natural cause,* z*, so if we had access to this* ***latent variable*** z*, generation is an easier task.* Notice for generating using normalizing flow above, all of the heavy lifting is done by function f_θ. This relieves f_θ a little bit! Because we are already giving it the *latent variable*. I do not want to dig into philosophical aspects of such latent information, so for now all that we know is that their presence makes the problem simpler as we can get away from the intractable integral in (4).

$$\log p_{\text{model}}(x) = \log \int p(x \mid z; \theta)\, p(z)\, dz \tag{5}$$

$$= \log \int f_\theta(z)\, p(z)\, dz \tag{6}$$

Now our model f_θ generates new samples given a latent variable z. In order to make sure marginalization over x in (5) adds up to one, we choose $f_\theta(z)$ as some probability distribution, like

$$p(x \mid z; \theta) = \mathcal{N}\big(x; f_\theta(z), \sigma^2 I\big).$$

You might wonder what has changed from before? The only thing that has changed is our perspective toward the problem. We now are assuming the generation task requires a complex inference and is hard to solve directly, so we are assuming there exists a latent variable where if it were given to us, the problem will be turned into a much simpler problem.

Yet, finding the latent variable as their name suggests is not an easy task. Each latent variable z_i captures the essence of the sample x_i, and claiming we found the essence is a big claim. Moreover, even if we find such latent variables the integral (5) is still intractable because it involves integration over all possible values of z which we do not know and how high-dimensional it is.

To mitigate the above, [2] proposed that we approximate z with a random variable Z having distribution $q(z) = \mathcal{N}(0, I)$. So basically each data point is mapped to samples of that distribution, thus obtaining codes following normal distribution. Each of these codes z_i hopefully would describe the essence of each sample x_i. This makes generation incredibly simple since we just need to feed the model f_θ some normally distributed z, thus obtaining samples following $p(x \mid z)$ distribution. Notice the assumption that codes z follow standard gaussian is not simplistic as normal distribution can be mapped to any distribution.

We talked a lot, so let's wrap up the discussion by setting up the objective. We are approximating the true latent posterior $p(z \mid x)$ using $q(z \mid x)$. Therefore, the objective can be written by minimizing the divergence between these two distributions:[1]

$$\mathrm{KL}\big(q(z \mid x) \,\|\, p(z \mid x)\big) = \mathbb{E}_{q(z|x)}\left[\log q(z \mid x)\right] - \mathbb{E}_{q(z|x)}\left[\log p(x, z)\right] + \log p(x) \tag{7}$$

Rearranging, we obtain the objective for VAEs, also known as Evidence Lower Bound (ELBO):

$$\log p(x) - \mathrm{KL}\big(q(z \mid x) \,\|\, p(z \mid x)\big) = \underbrace{\mathbb{E}_{q(z|x)}\left[\log p(x \mid z)\right] - \mathrm{KL}\big(q(z \mid x) \,\|\, p(z)\big)}_{\text{ELBO}} \tag{8}$$

So optimizing the right-hand side essentially increases $\log p(X)$, because the right hand side is a lower bound for our likelihood. The first term, since $p(x \mid z)$ is normally distributed, translates to reconstruction and the second term enforces the codes to follow standard gaussian. In very high-level what is happening is that we

[1] The objective can also be derived by expanding equation (5) by multiplying and dividing $q(z|X)$ and using Jensen equality.

learn an encoder model $q(z \mid x)$ that encodes observation x_i, and we enforce this codes to become normally distributed codes, and simultaneously a decoder $f_\theta(z;\theta)$ is trying to decode such codes.

When we want to maximize the lower bound by optimizing parameters of $p(x|z)$ or equivalently f_θ(remember $p(x|z) = \mathcal{N}(f_\theta(z), \sigma^2 I)$) we encounter a problem due to sampling from $q(z|x)$, which itself depends on parameters we wish to optimize[3]. To address this, we employ *reparameterization trick*. Specifically, we represent samples from $q(z|x)$ using a deterministic transformation:

$$z = \mu(x) + \sigma(x) \odot \epsilon, \quad \text{where} \quad \epsilon \sim \mathcal{N}(0, I) \tag{9}$$

This enables gradients to flow through sampling operations, thus enabling backprop for our model.

Diffusion Models as Markovian Hierarchical VAEs

In VAEs we break our generation procedure into inferring a single latent code and then decoding from that code. But why stop at only one latent? That's where hierarchical VAEs come into play. Hierarchical VAEs introduce a sequence of latent variables linked in a Markovian chain. The way that I like why we need them is that, the ELBO always lower-bounds the true log-likelihood by exactly $\mathrm{KL}\big(p(z \mid x) \,\|\, q(z \mid x)\big)$, so by stacking more latent variables we can progressively tighten this bound. Concretely, with T latents $\{z_1, \ldots, z_T\}$ arranged in a Markovian hierarchy, z_T—being furthest from the data distribution[2]—matches a simple prior (and is therefore easy to sample), whereas z_1 sits closest to x and requires the most expensive inference.

$$q(z_{1:T} \mid x) = q(z_1 \mid x) \prod_{t=2}^{T} q(z_t \mid z_{t-1}), \tag{10}$$

$$p(x, z_{1:T}) = p(x \mid z_1) \left[\prod_{t=2}^{T} p(z_{t-1} \mid z_t) \right] p(z_T). \tag{11}$$

Importantly, we do not have learnable encoders any more. We set next state of latent variable as a noised version with some scheduler α_t, specifically $q(z_t \mid z_{t-1}) = \mathcal{N}\big(z_t; \sqrt{\alpha_t}\, z_{t-1}, (1-\alpha_t)I\big)$, and each step in this Markovian chain is called *forward process*. Moreover, we choose the prior $p(z_T) = \mathcal{N}(0, I)$ for easy sampling. We then learn the *backward process*—a family of denoising kernels $p(z_{t-1} \mid z_t)$. Once these backward transitions are learned, we generate new samples by starting from z_T and iteratively applying $p(z_{t-1} \mid z_t)$ down to z_1, finally decoding z_1 to x. This ancestral sampling procedure recovers samples from p_{data}.

We can learn the backward steps proceeds similar to VAEs by minimizing an ELBO. However, if we write that ELBO naively, the Monte Carlo estimate suffers from high variance and therefore needs many samples for an accurate gradient [4]. The trick here is to *condition on the trajectory's end-points*. Because the latent variables are Markovian, fixing the starting point x_0 does not effect our forward distribution(keep this in mind—it becomes invaluable in flow matching!). Conditioning and rearranging yields the diffusion-model objective [4].

$$\begin{aligned} \mathcal{L}_{\text{DM}} \;=\; & \mathbb{E}_{q(x)}[-\log p(x \mid z_1)] + \mathrm{KL}(q(z_T \mid x) \,\|\, p(z_T)) \\ & + \sum_{t=2}^{T} \mathbb{E}_{q(z_t|x)}[\mathrm{KL}(q(z_{t-1} \mid z_t, x) \,\|\, p(z_{t-1} \mid z_t))] \end{aligned} \tag{12}$$

We can simply learn parameters of $p(z_t \mid z_{t-1})$ by matching forward distribution to backward counterpart. Thanks to the conditioning step, the exact forward term $q(z_{t-1} \mid z_t, x_0)$ is Gaussian[3], so with a variance-preserving schedule the model only needs to predict the mean of latent variables since both forward and backward are Gaussians with the same variance. Replacing each KL in (12) by a mean-squared-error between posterior and predicted means gives the loss used in DDPMs:

$$\mathcal{L}_{\text{MSE}} = \sum_{t=1}^{T} w_t \, \mathbb{E}_{q(x_0, z_t)} \, \|\mu_\theta(z_t, t) \;-\; \mu_q(z_t, x_0)\|_2^2 \tag{13}$$

[2]Distance can be precisely defined by introducing a metric on the space of probability measures, for instance KL divergence
[3]Gaussian assumption for backward is only valid if we take sufficiently small steps .

where $\mu_q(z_t, x_0) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha_{t-1}})z_t + \sqrt{\bar{\alpha_{t-1}}}(1-\alpha_t)x_0}{1-\bar{\alpha_t}}$ is the closed-form for our latent variable mean, and w_t are optional weighting factors. Minimizing (13) teaches the network to denoise each latent step. Assuming that the model learned backward process, the samples can be generated simply from ancestral sampling as mentioned earlier.

Notice in practice we train a *single* neural network, not T separate ones. At every SGD step we draw a data sample $x_0 \sim p_{\text{data}}$ and a timestep $t \sim \mathcal{U}\{1, \ldots, T\}$, corrupt x_0 to z_t with the forward transition kernel $q(z_t \mid x_0)$, feed (z_t, t) (the time is passed through a sinusoidal or learned embedding) to the network, and minimize the weighted MSE objective in (13).

Moreover, Instead of predicting the original image using $\mu_q(z_t, x_0)$, we can equivalently regress the injected noise ϵ. In [5], they showed that this variant yields better image quality. Additionally, by Tweedie's formula the optimal noise predictor, by mean square error sense, is proportional to the score $\nabla_{z_t} \log q(z_t)$; hence DDPMs implicitly learn the score of the perturbed data distribution, changing our perspective toward diffusion models as score predictors [4].

Flow Models

So far, we examined hierarchical latent variable models where each latent variable depends only on the previous step. This framework can be generalized to a continuous form, where the process can be interpreted as a continuous-time Markov process. The generation process involves starting from some prior distribution $p_T \sim p_{\text{init}}$ and then taking backward steps (similar to diffusion models, but in a continuous manner) to reach the target distribution $p_0 \sim p_{\text{data}}$.

To understand flow models, we can build on top of our existing knowledge. Consider the space of all probability measures $\mathcal{P}(\mathcal{X})$ on our input space $\mathcal{X}$. Each point in this space corresponds to a probability measure that assigns weights to points in the input space according to its density.

In this space, all probability distributions can be found, including the standard Gaussian and, more importantly, *the infamous data distribution* p_{data}. From this perspective, what we were doing with diffusion models can be interpreted as starting with some known distribution p_{init} and traversing this space to reach p_{data}.

However, moving in this infinite-dimensional space is challenging. To simplify, let us define that we are at p_{init} at time $t = 0$ and wish to reach p_{data} at time $t = 1$. If we knew the infinitesimal steps toward p_{data}, then we could move along this path. So, we start from p_0 (which we considered a Gaussian) and take one infinitesimal step forward. To define this step, we have to relate it to samples in our input space - because all we have is some observations after all. Since the probability distributions in measure space assign a mass to each point in input space with a specific value corresponding to its density, and considering the fact that total mass in the space should be one at all times*(conservation of mass)*, we can use the continuity equation to find the probability density after an infinitesimal step forward. Basically, if we consider a closed region, the change in probability distribution can be obtained by total mass coming in and going out. The pointwise interpretation of it can be as eq (14).

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot (p_t(x) u_t(x)) \tag{14}$$

In (14), $u_t(x)$ represents the velocity field at time t. This equation demonstrates that points in the space are being "pushed" in different directions by this vector field. If there were no vector field, the probability would remain constant for all time, so there **must** be a vector field.

Thus far, we have established that if we knew how to "push" the points in the space for all time $t \in [0, 1]$, we could solve the ordinary differential equation (ODE) and generate samples from the data distribution. So the problem boils down to solving regression problem in eq (15) :

$$\min_{\theta} \mathbb{E}_{t \sim \mathcal{U}[0,1], x_t \sim p_t} \left[\|u_\theta(x_t, t) - u_t(x_t)\|^2 \right] \tag{15}$$

However, solving (15) is not possible as we do not even have the targets $u_t(x_t)$ for our model.

Our approach to move forward from this dead end is to define a notion of conditional vector fields. These are obtained by finding the vector field that, when simulated, allows us to start from the initial distribution p_0 and collapse to a single point on which we condition(recall what we did for diffusion models, we conditioned on starting point, we do the same here). So rewriting our objective gives us eq. (16).

$$\min_{\theta} \mathbb{E}_{x_0 \sim p_{data}, t \sim \text{Unif}, x_t \sim p_t(\cdot|x_0)}[\|u_t^{\theta}(x_t) - u_t(x_t|x_0)\|^2] \tag{16}$$

You might wonder why we can use optimization problem (16), which is doable instead of (15)? It can be proven that with respect to our optimization parameter θ these two are the same optimization problem [6]. Fortunately, obtaining the conditional vector field $u_t(x_t|x_0)$ is easy, and it's actually our design choice. More specifically, we might prefer certain types of paths from others, like linear paths, to generate samples faster during inference [7].

Guidance

So far, we have discussed generative models that sample from an unconditional distribution $p(x)$. However, in many practical applications, we want to control the generation process by conditioning on additional information, generating image that describes a specific text prompt or a class label. This conditioning process is referred to as guidance[4]. Formally, for a conditional variable y from some space $\mathcal{Y}$, we aim to sample from $p_{\text{data}}(x|y)$ rather than just $p_{\text{data}}(x)$.

When implementing guidance in flow models, we can modify our objective function to account for the conditional distribution[6]. The guided conditional flow matching objective becomes:

$$\min_{\theta} \mathbb{E}_{(x_0,y)\sim p_{\text{data}}(x_0,y), t\sim \text{Unif}[0,1), x_t \sim p_t(\cdot|x_0)}[\|u_t^{\theta}(x_t|y) - u_t(x_t|x_0)\|^2] \tag{17}$$

Here, the neural network $u_t^{\theta}(x_t|y)$ is trained to approximate the conditional vector field while being guided by y. While this approach theoretically produces samples from $p_{\text{data}}(\cdot|y)$, empirical evidence showed that generated samples often did not adhere closely enough to the guiding variable y [6]. To address this limitation, classifier-free guidance was introduced as a technique to increase the effect of the guiding variable. The key insight involves using Bayes rule to decompose the guided score function, allowing us to scale up the contribution of the guidance term.

$$\tilde{u}_t(x|y) = (1-w)u_t(x|\emptyset) + wu_t(x|y) \tag{18}$$

During inference, since we only have one model only we define a special null token $\emptyset$ representing the absence of conditioning, and combine the guided and unguided vector fields. where $w > 1$ is the guidance scale that controls adherence to the condition. For $w = 1$, we recover the original guided vector field, while larger values of w place more emphasis on the conditioning variable y.

For training with classifier-free guidance, we introduce an additional hyperparameter η representing the probability of replacing the original label y with the null token $\emptyset$. The resulting classifier-free guidance conditional flow matching objective becomes[6]:

$$\min_{\theta} \mathbb{E}_{\square}[\|u_t^{\theta}(x|y) - u_t^{\text{target}}(x|z)\|^2] \tag{19}$$

where $\square = \{(z,y) \sim p_{\text{data}}(z,y), t \sim \text{Unif}[0,1), x \sim p_t(\cdot|z), \text{replace } y = \emptyset \text{ with prob. } \eta\}$. This approach not only improves sample quality and adherence to the guiding variable but also allows for parameter sharing between conditional and unconditional models, reducing memory requirements and providing flexibility during inference through adjustable guidance scales.

[4]We focus on guidance in flow matching literature only here.

References

[1] S. J. Prince, *Understanding Deep Learning*. The MIT Press, 2023.

[2] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2022.

[3] C. Doersch, "Tutorial on variational autoencoders," 2021.

[4] C. Luo, "Understanding diffusion models: A unified perspective," 2022.

[5] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," 2020.

[6] P. Holderrieth and E. Erives, "Introduction to flow matching and diffusion models," 2025.

[7] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," 2023.